

Тестирование идиempотентности и повторяемости запросов в REST API

по дисциплине «Методы тестирования программного обеспечения»

Плахотников Владимир Александрович

СПбПУ, ИКНТ ВШ ПИ

2026

Выполнил студент группы 5130903/30303:
Плахотников Владимир Александрович

Руководитель:
Старший преподаватель В. А. Пархоменко

Структура доклада

- 1 Постановка проблемы
- 2 Математическая формализация
- 3 Алгоритм
- 4 Три типа контроллеров
- 5 Иллюстративный пример
- 6 Апробация
- 7 Выводы

Проблема: При сетевых сбоях клиент повторяет HTTP-запрос (retry).

Без идемпотентности:

- POST-запрос на создание заказа $\times 3$ retry $\Rightarrow$ 3 заказа
- POST-запрос на платёж $\times 3$ retry $\Rightarrow$ **тройное списание**
- POST товара с тем же SKU $\Rightarrow$ дубликат в каталоге

Цель: Исследовать и продемонстрировать тестирование идемпотентности REST API с тремя различными типами контроллеров Spring Boot.

Определение. Операция $O : S \rightarrow S$ *идемпотентна*, если:

$$O^n(s) = O(s), \quad \forall n \geq 1, \forall s \in S$$

HTTP-методы (RFC 9110):

Метод	Safe	Идемпотентный
GET, HEAD	✓	✓
PUT, DELETE	—	✓
POST, PATCH	—	—

Решение для POST: Idempotency-Key k :

$$O'(s, r, k) = \begin{cases} O(s, r), & k \notin \text{KeyStore} \\ \text{cached}(k), & k \in \text{KeyStore} \end{cases}$$

Алгоритм 1 CheckIdempotency(*request*)

```
1: если request.method  $\neq$  POST тогда
2:   вернуть ProcessRequest(request)
3: конец если
4: key  $\leftarrow$  headers["Idempotency-Key"]
5: если key =  $\emptyset$  тогда
6:   вернуть 400 Bad Request
7: конец если
8: cached  $\leftarrow$  KeyStore.find(key)
9: если cached  $\neq \emptyset$  и cached.response  $\neq \emptyset$  тогда
10:  вернуть cached.response
11: конец если
12: KeyStore.register(key)
13: response  $\leftarrow$  ProcessRequest(request)
14: KeyStore.save(key, response)
15: вернуть response
```

▷ кэшированный ответ

Три кардинально разных типа контроллеров Spring

	@RestController	Functional (WebMvc.fn)	Spring Data REST
Домен	Заказы	Платежи	Товары
Стиль	Аннотации	Функциональный	Автогенерация
Маршрутизация	@GetMapping	RouterFunction	Автоматическая
Идемпотентность	Idempotency-Key	Idempotency-Key	@Version + ETag
Источник	Spring Guides	Spring Framework docs	Spring Data REST Guide

Ключевое отличие: способ определения маршрутов и обработчиков принципиально различается, что влияет на подход к обеспечению идемпотентности.

Иллюстративный пример: retry платежа

Сценарий: Клиент отправляет POST /api/payments, получает таймаут, повторяет запрос.

С идемпотентностью (Idempotency-Key: "pay-001"):

- 1 Запрос #1: key="pay-001" $\notin$ KeyStore $\Rightarrow$ register, создать платёж, сохранить ответ
- 2 Запрос #2 (retry): key="pay-001" $\in$ KeyStore $\Rightarrow$ **вернуть кэш**
- 3 Результат: **1 платёж** (корректно)

Без идемпотентности:

- 1 Запрос #1: создать платёж $\Rightarrow$ списание #1
- 2 Запрос #2 (retry): создать платёж $\Rightarrow$ списание #2
- 3 Результат: **2 платежа — двойное списание!**

Пошаговое выполнение на данных

Вход: POST /api/orders, body: {description: "Ноутбук", amount: 89999.99}

Шаг	Действие	KeyStore	orders (count)
0	Начальное состояние	∅	0
1	Получен key="abc"	∅	0
2	find("abc") = ∅	∅	0
3	register("abc")	{abc: pending}	0
4	INSERT order	{abc: pending}	1
5	save("abc", 201, body)	{abc: 201, body}	1
6	Retry: key="abc"	{abc: 201, body}	1
7	find("abc") ≠ ∅	{abc: 201, body}	1
8	return cached 201	{abc: 201, body}	1

Результат: 1 заказ, 2-й запрос вернул кэш. Идемпотентность обеспечена.

Технологии: JUnit 5 + Testcontainers (PostgreSQL в Docker) + MockMvc

Тестовый класс	Тестов	Профиль	Результат
OrderIdempotencyTest	6	idempotent	PASS
PaymentIdempotencyTest	5	idempotent	PASS
ProductIdempotencyTest	5	idempotent	PASS
DataImportTest	2	idempotent	PASS
NonIdempotentProfileTest	4	non-idempotent	PASS*
Итого	22		PASS

*NonIdempotentProfileTest: тесты проходят, **подтверждая наличие багов** при отключённой идемпотентности (дубликаты, тройное списание).

Профиль non-idempotent: демонстрация проблем

Тест	Проблема	Статус
POST без Idempotency-Key	Проходит без проверки	BUG
Повторный POST (тот же key)	Дубликат заказа	BUG
Дубликат SKU товара	Проходит (нет constraint)	BUG
3x retry платежа	Тройное списание	BUG

Ключевой вывод: отключение механизмов идемпотентности приводит к критическим багам, обнаруживаемым тестами.

- 1 Формализована математическая модель идемпотентности HTTP: $O^n(s) = O(s)$
- 2 Реализованы 3 типа контроллеров Spring с разными механизмами идемпотентности:
 - @RestController + Idempotency-Key
 - Functional Endpoints + Idempotency-Key
 - Spring Data REST + @Version/ETag
- 3 22 интеграционных теста (Testcontainers + PostgreSQL)
- 4 Профиль non-idempotent демонстрирует: без идемпотентности — дубли и тройное списание

Спасибо за внимание!